

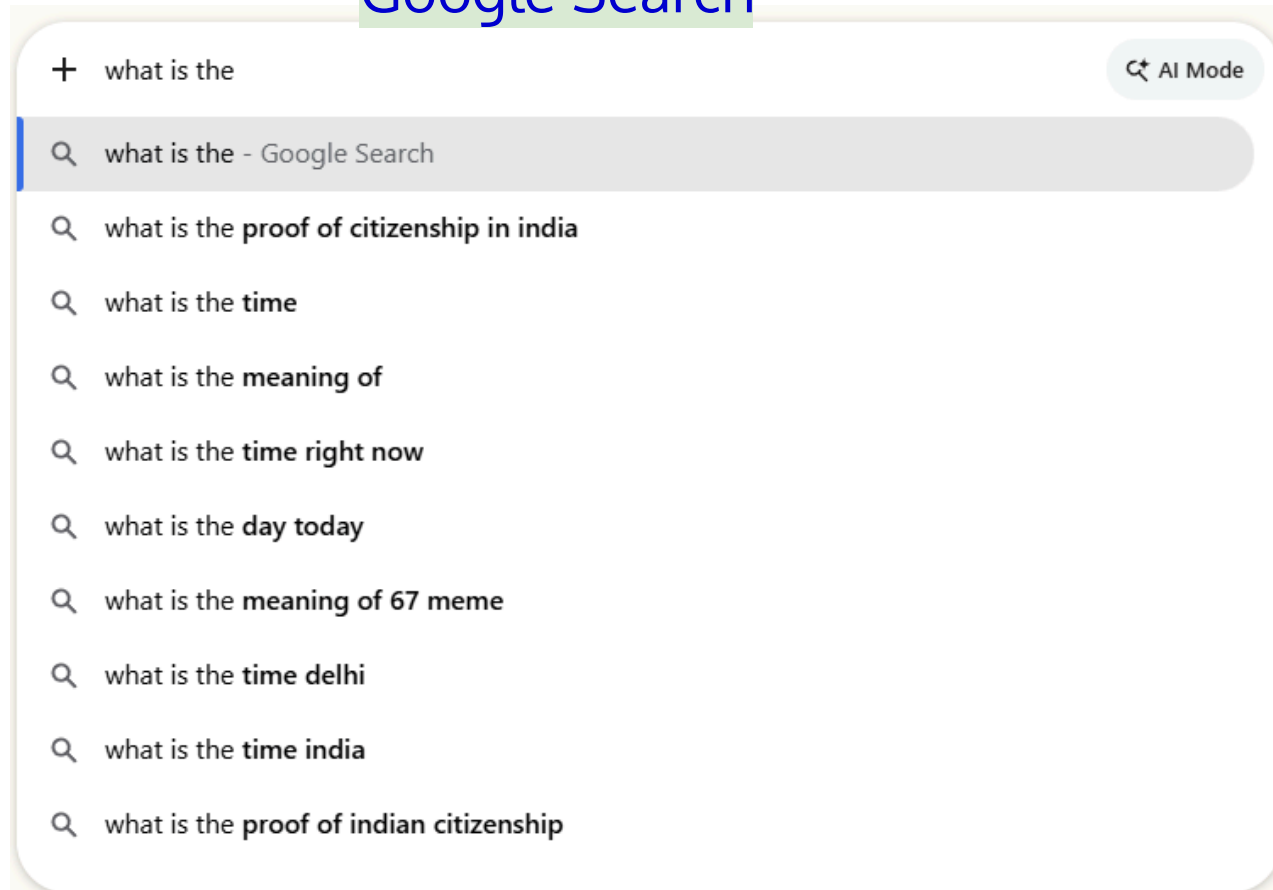
Language Modeling

From RNNs to Attention

Language Modeling

We use Language Models every day!

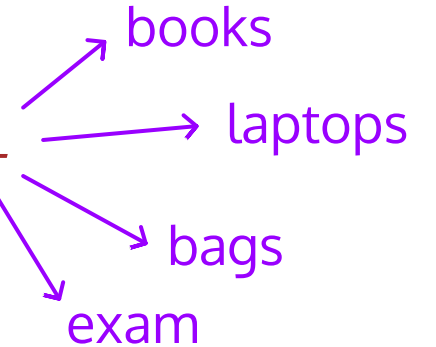
Google Search



A language model predicts what comes next

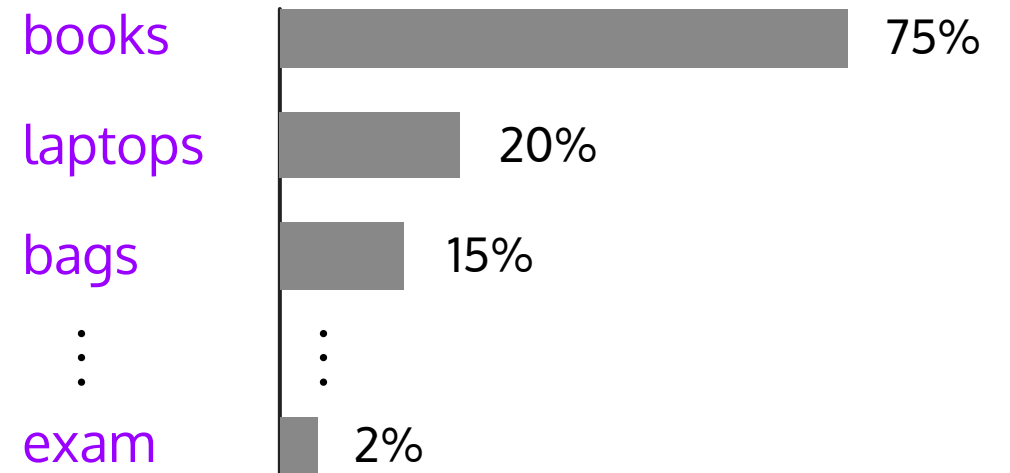
- Given some words, a Language Model (LM) predicts the **most likely next token**

Example: *"the students opened their _____"*



books
laptops
bags
exam

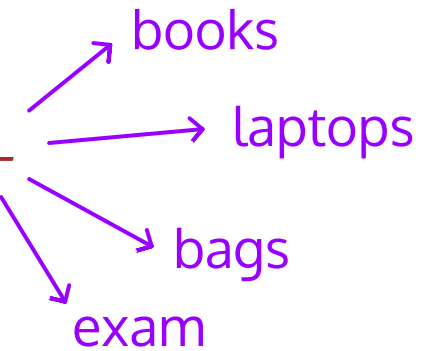
- After seeing *"the students opened their _____"*, the LM assigns probabilities to all tokens in its vocabulary:



A language model predicts what comes next

- Given some words, a Language Model predicts the **most likely next token**

Example: *"the students opened their _____"*



- books
- laptops
- bags
- exam

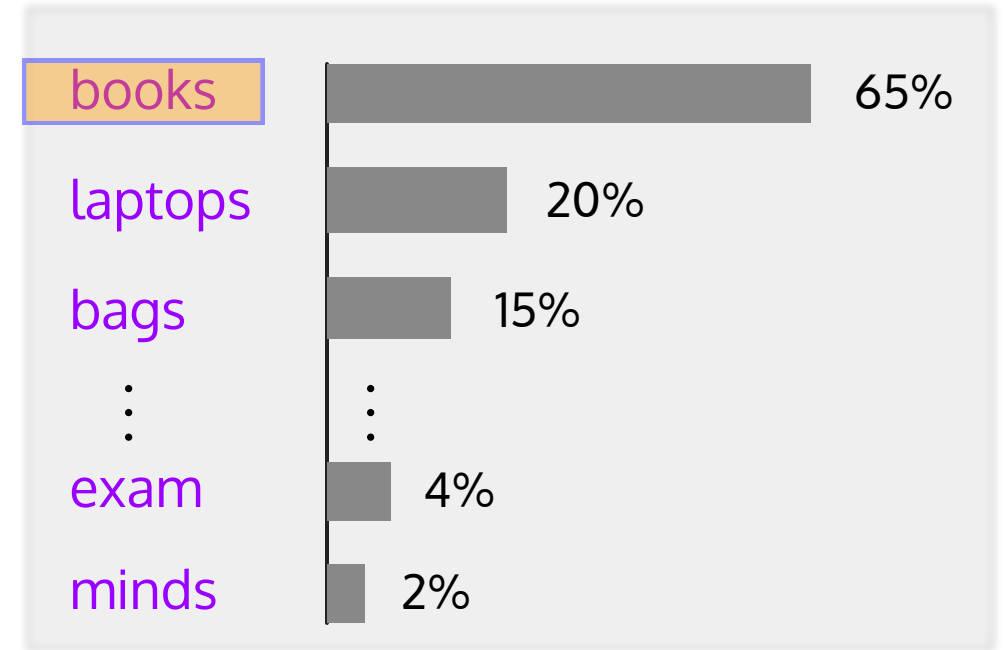
- First, create a **vocabulary** of all words the model knows
- Then, after reading a sequence of tokens $x_1, x_2, \dots, x_t$ produced using the vocabulary, the model gives a probability to every possible next token x_{t+1}

$$P(\text{next token} \mid \text{tokens seen so far}) \xrightarrow{\text{mathematically}} P(x_{t+1} \mid x_t, \dots, x_1)$$

Sampling tokens

- After seeing "*the students opened their _____*", the LM assigns token probabilities:

- Sampling strategies are used to choose a token
 - Naive strategy → Select token with the highest probability

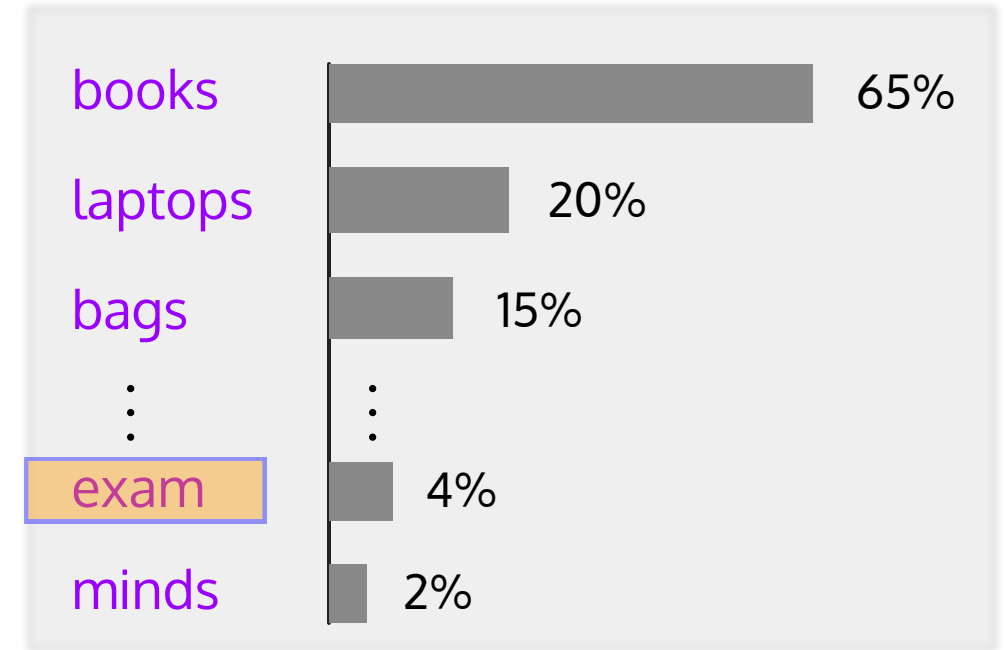


"the students opened their books"

A language model predicts what comes next

- After seeing *"the students opened their _____"*, the LM assigns token probabilities:

- Sampling strategies are used to choose a token
 - Naive strategy: Select token with the highest probability
 - Another strategy: Randomly sample using probability

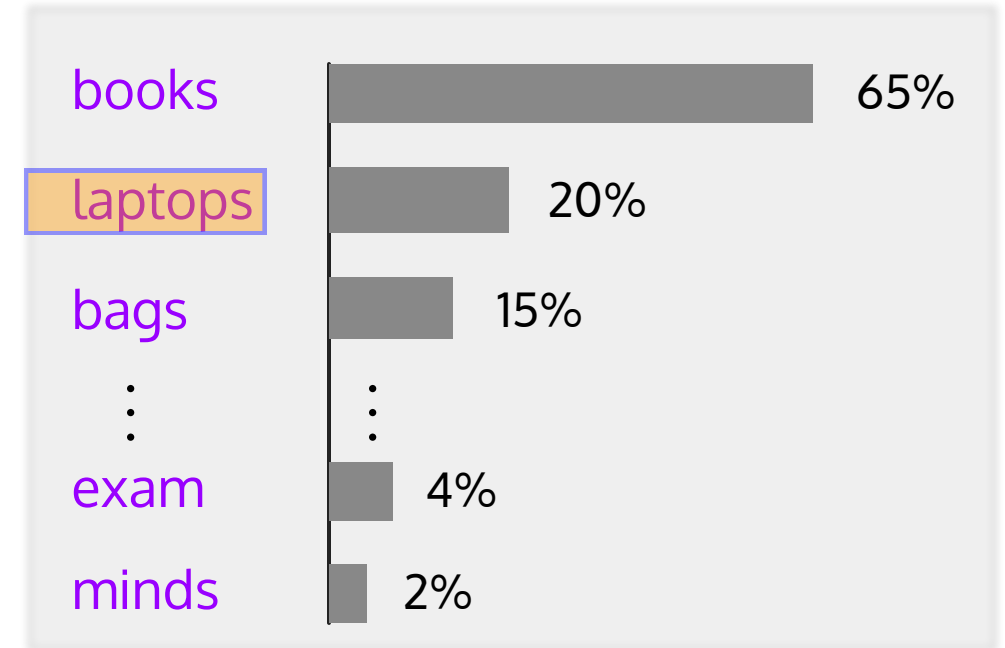


"the students opened their exams"

A language model predicts what comes next

- After seeing "*the students opened their _____*", the LM assigns token probabilities:

- Sampling strategies are used to choose a token
 - Naive strategy: Select token with the highest probability
 - Another strategy: Randomly sample using probability
 - Another strategy: Randomly sample from top-k tokens



"the students opened their laptops"

A language model predicts at every position

- **Training** creates a next-token prediction problem at every position

	Inputs seen so far	Target next token	
Step 1	<User query>	the	$P(x_1)$
Step 2	the	students	$P(x_2 \mid x_1)$
Step 3	the students	opened	$P(x_3 \mid x_2, x_1)$
Step 4	the students opened	their	$P(x_4 \mid x_3, x_2, x_1)$
Step 5	the students opened their	laptops	$P(x_5 \mid x_4, x_3, x_2, x_1)$

* During training, the correct next token is known

Training a language model

User Query (Prompt)

The students opened their

Tokenizer

Tokenization (break down text into small words or sub-words)

Tokens

the students opened their

Embedding (turning tokens into dense vectors capturing their meaning)

Embeddings



Language Model

(process the text and predicts the next token)

Ingredients for training a language model

User Query (Prompt)

What is the outcome of the inspection?

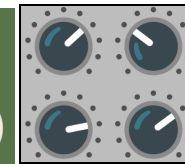
⋮

Embeddings



LM parameters

Language Model
(process the text and predicts the next token)



Optimize with backprop

Token probability

(scores all tokens in the vocabulary using **softmax**)

laptops

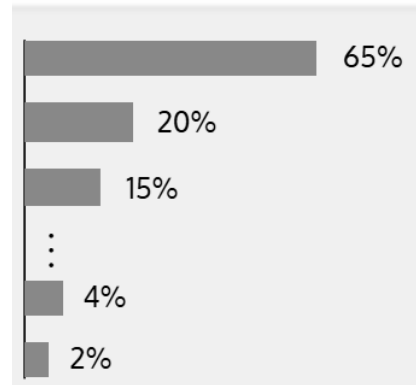
books

exams

⋮

bags

minds



$\hat{y}_1 = \text{"laptops"}$

$y_1 = \text{"books"}$

Actual token

Cross-entropy loss

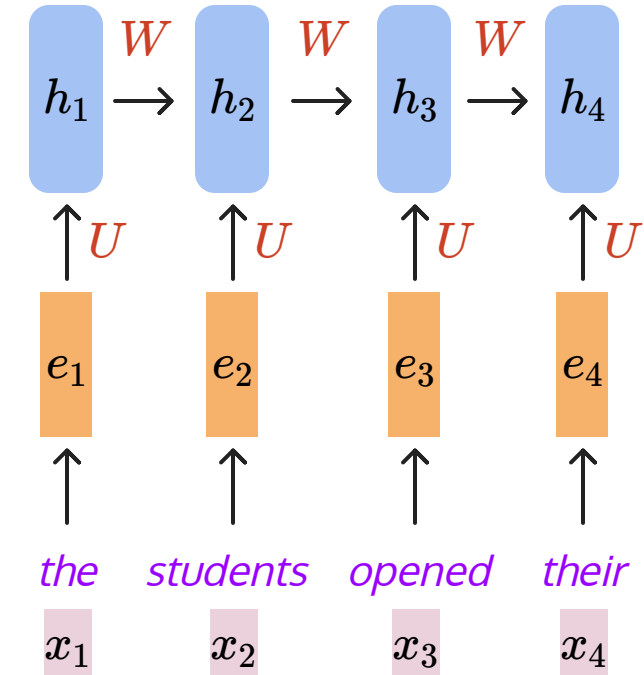
Sampling strategies are used to choose a token

RNN as a language model

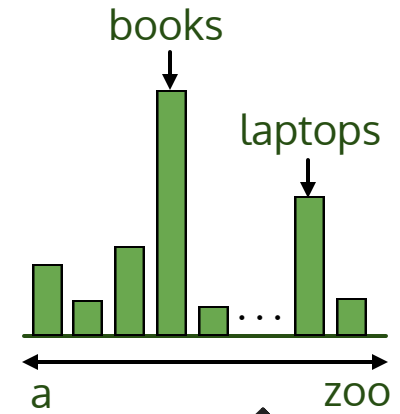
- An RNN can perform language modeling
- An RNN language model carries context **forward**

$$\underset{\text{output}}{y_t} = f(\underset{\text{input}}{x_t}, \underset{\text{past memory}}{h_{t-1}}; W, U, V, b, c)$$

- At each step, the RNN combines:
 - the current token x_t
 - the memory carried from earlier tokens h_{t-1}
- It then predicts a probability distribution for the next token



$P(\hat{y}_4 \mid \text{the students opened their})$



The central long-context question

- What if the crucial clue appears much earlier?
- Can an RNN always retain the right piece of information until it is needed?
- The hidden state can often dilute information for long sequences
- Because information flows in one direction -> Bidirectional RNNs

[https://www.youtube.com/embed/v2bGrF0FmF0?
enablejsapi=1](https://www.youtube.com/embed/v2bGrF0FmF0?enablejsapi=1)

RNN language models work – but only for one kind of task

What works — continuation

- Input and output are the same sequence
- Each next token continues what came before
- The hidden state carries everything needed

"The bridge was" → RNN LM → "inspected"

What breaks — different sequences

Long inspection report → ? → Short safety summary

- The output is a different sequence, not a continuation
- The model must read one thing and write another
- No mechanism exists to separate reading from generating

What if the model could read one sequence and then generate a completely different one?

Language Translation task

New task type: Sequence-to-sequence modeling

Input: A sequence

"पेरिस में मौसम गर्म है"



RNN Language Model



Output: Another sequence of possibly different length

"The weather in Delhi is hot"

| Transform one sequence into another

Encoder-decoder RNNs

Takes a sequence as input and produces another sequence as output (also known as **seq2seq** model)

French - detected

je ne veux pas travailler

A

English

I do not want to work

B

Visual Chatbot

A'





Caption: a group of people standing around a room with a large tv

A

How many people?



there are too many to count

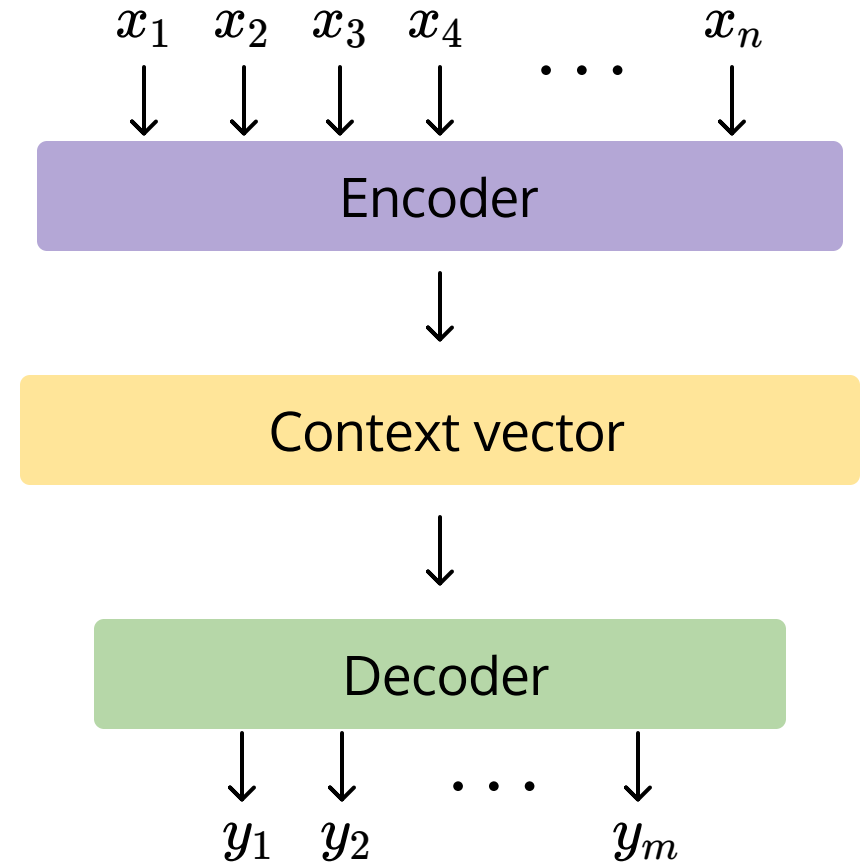
B

A

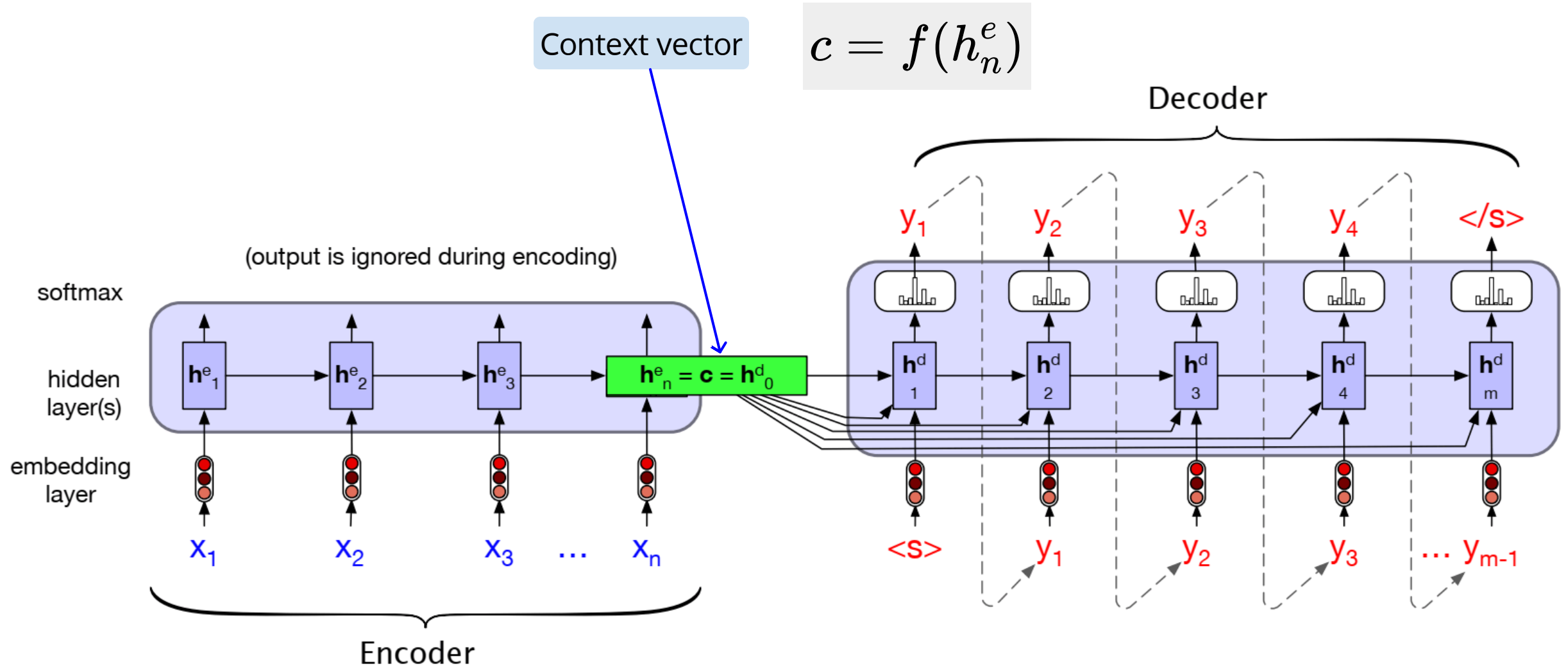
What are they wearing?

Encoder-decoder architecture

- **Encoder:** An NN that takes reads the input sequence and creates a representation for each input token
- **Context:** A vector collects the important information from the input and passes it to the decoder (*summary of input*)
- **Decoder:** An LM uses that context to generate a new output sequence, one token at a time

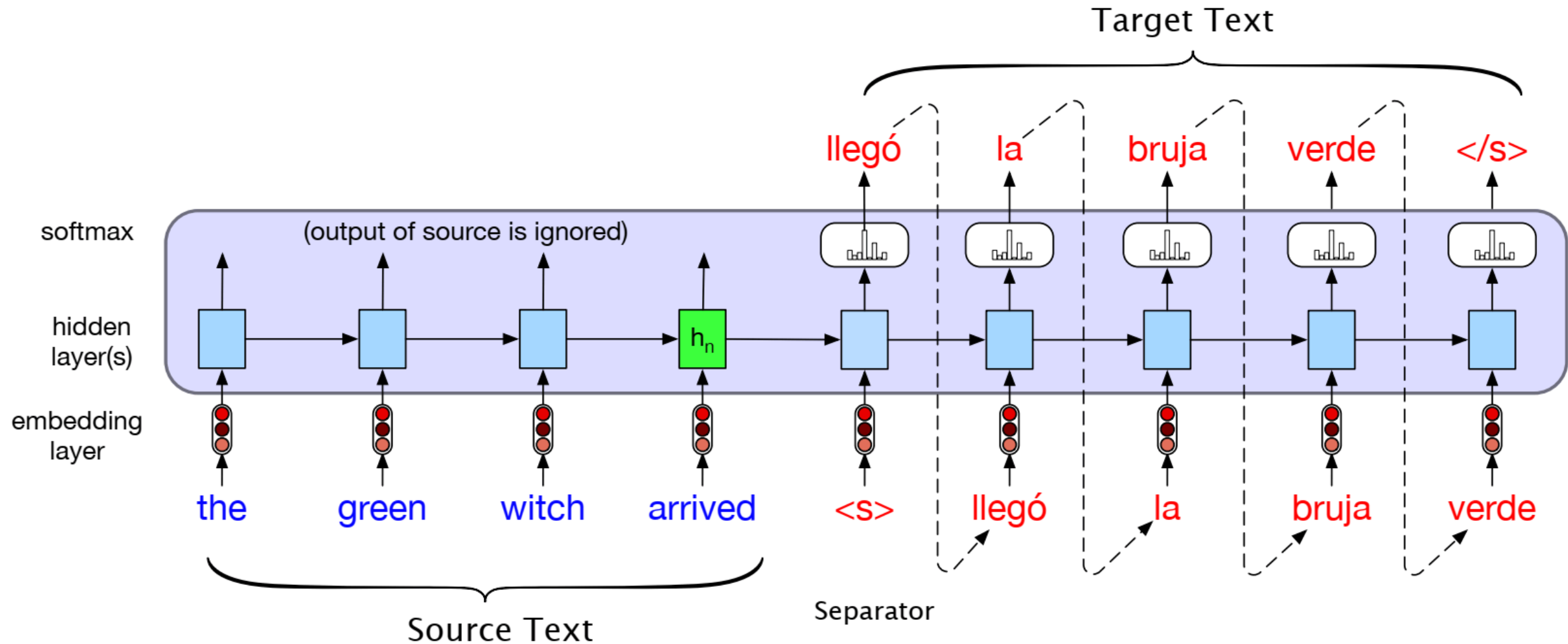


Encoder-decoder RNNs



Encoder-decoder RNN

Example: Neural Machine Translation

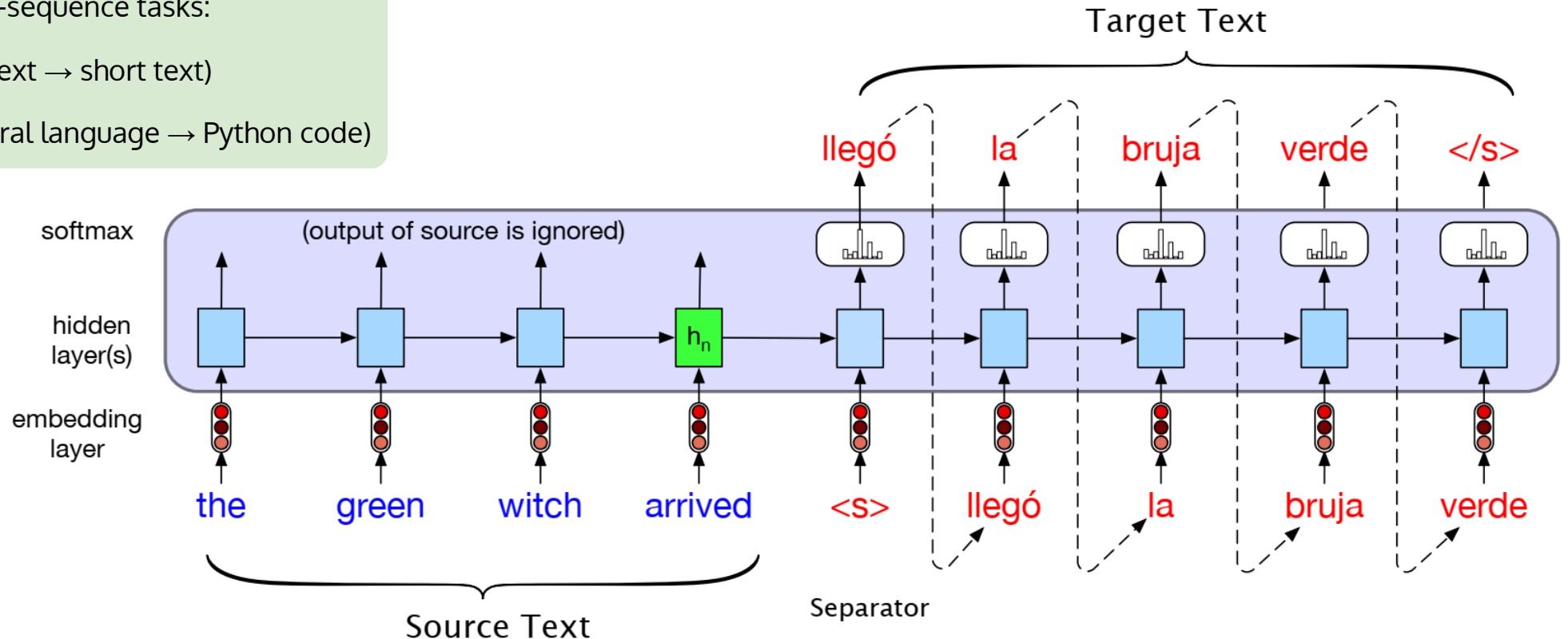


Encoder-decoder RNN

- The **sequence-to-sequence** model is an example of a **Conditional Language Model**
- **Language Model** because the decoder is predicting the next word of the target sentence
- **Conditional** because its predictions are also conditioned on the source sentence

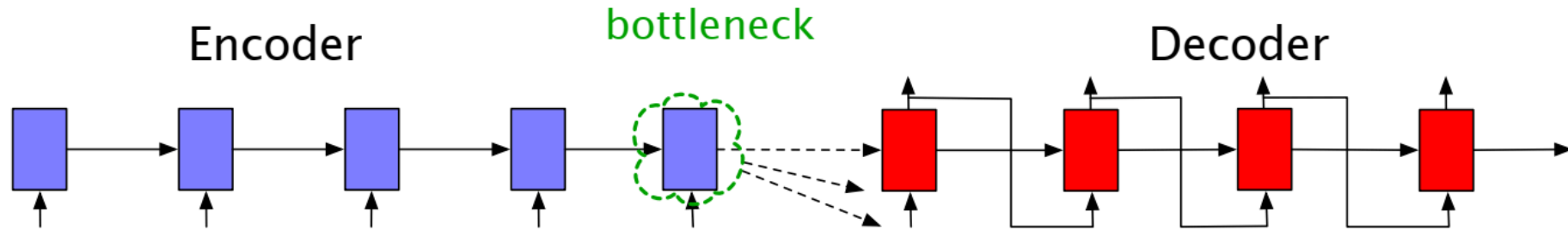
Some other sequence-to-sequence tasks:

- **Summarization** (long text → short text)
- **Code generation** (natural language → Python code)



Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough



- **One final context vector:** the encoder compresses the entire input sequence into a single representation
- **Decoder sees only this summary:** it cannot directly revisit the original input words
- **Long inputs are difficult:** important details from the beginning may be weakened, lost, or poorly represented

Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough

Input: Severe corrosion was found in the main bridge support

Output: Close the bridge immediately

When generating "Close",
the decoder especially needs: severe corrosion — danger signal triggers the imperative

Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough

Input: Severe corrosion was found in the main bridge support

Output: Close the bridge immediately

When generating "the",
the decoder especially needs: grammatical filler — minimal input focus needed

Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough

Input: Severe corrosion was found in the main bridge support

Output: Close the bridge immediately

When generating "**bridge**",
the decoder especially needs: main bridge support — the specific structure at risk

Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough

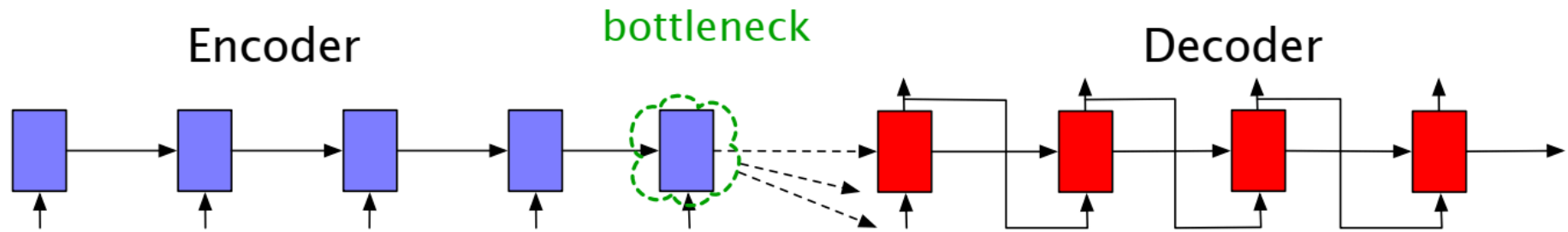
Input: Severe corrosion was found in the main bridge support

Output: Close the bridge immediately

When generating "**immediately**",
the decoder especially needs: severity + finding — urgency comes from how bad it is

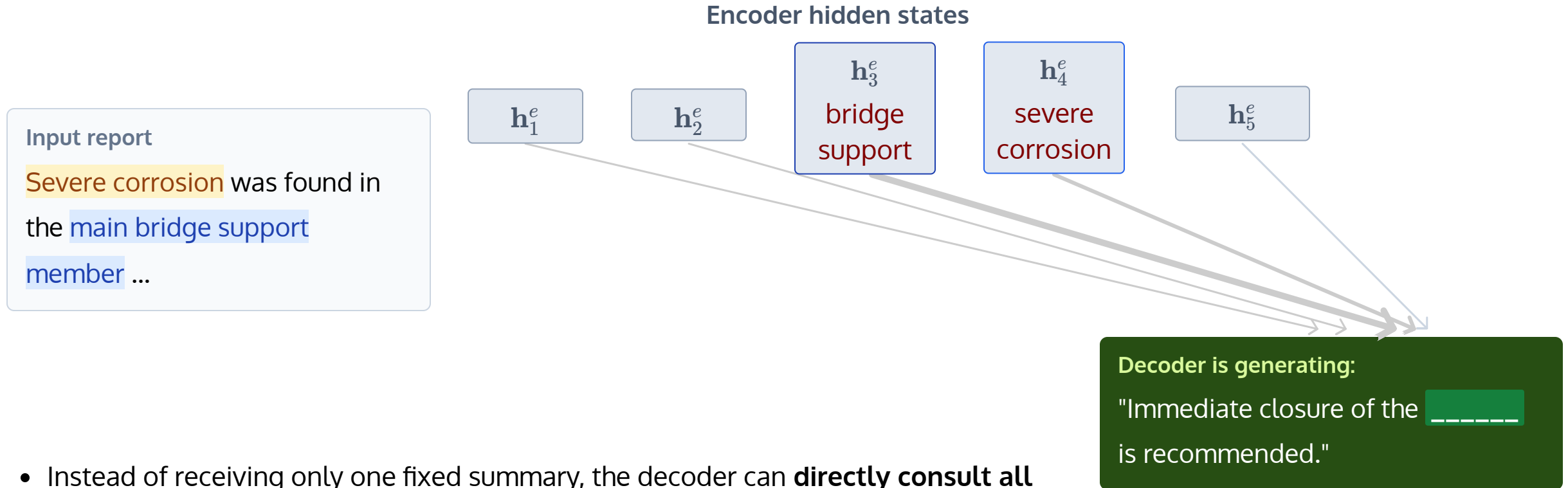
Limitations of Encoder-decoder RNN

One fixed-length context vector is not enough



- **One final context vector:** the encoder compresses the entire input sequence into a single representation
- **Decoder sees only this summary:** it cannot directly revisit the original input words
- **Long inputs are difficult:** important details from the beginning may be weakened, lost, or poorly represented

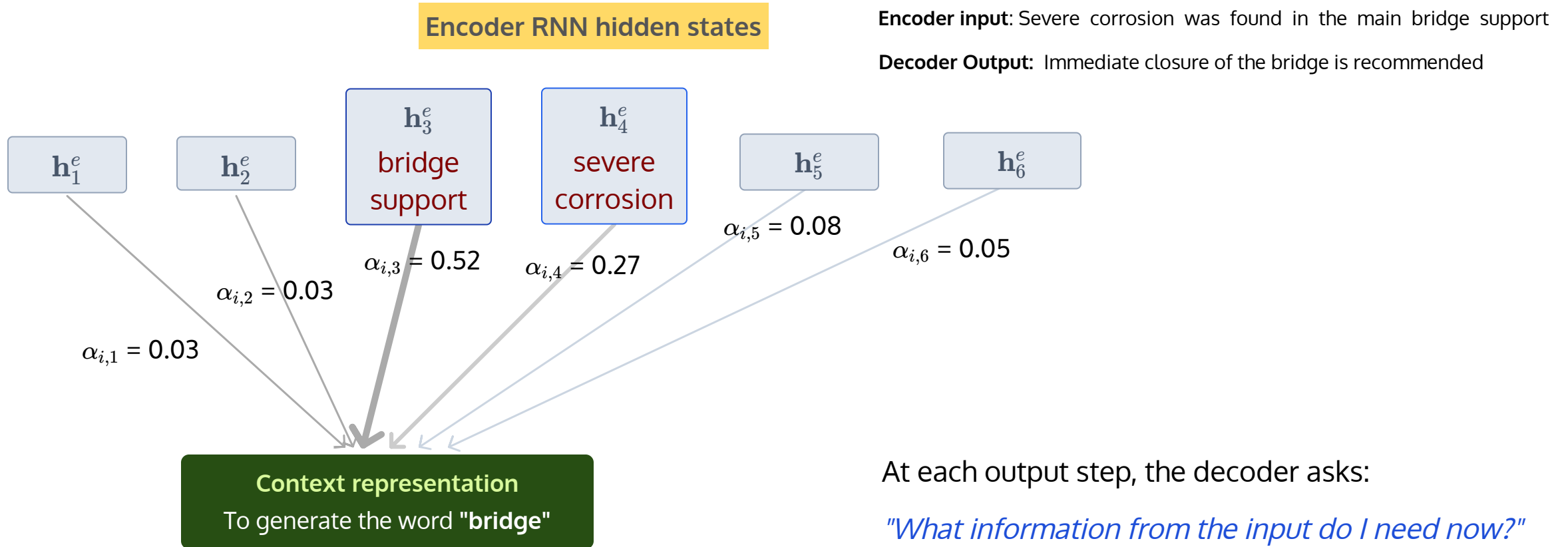
What if the decoder could look back at the input?



- Instead of receiving only one fixed summary, the decoder can **directly consult all encoder states**
- It decides which parts of the input matter for the output token it is generating now.

This is attention!

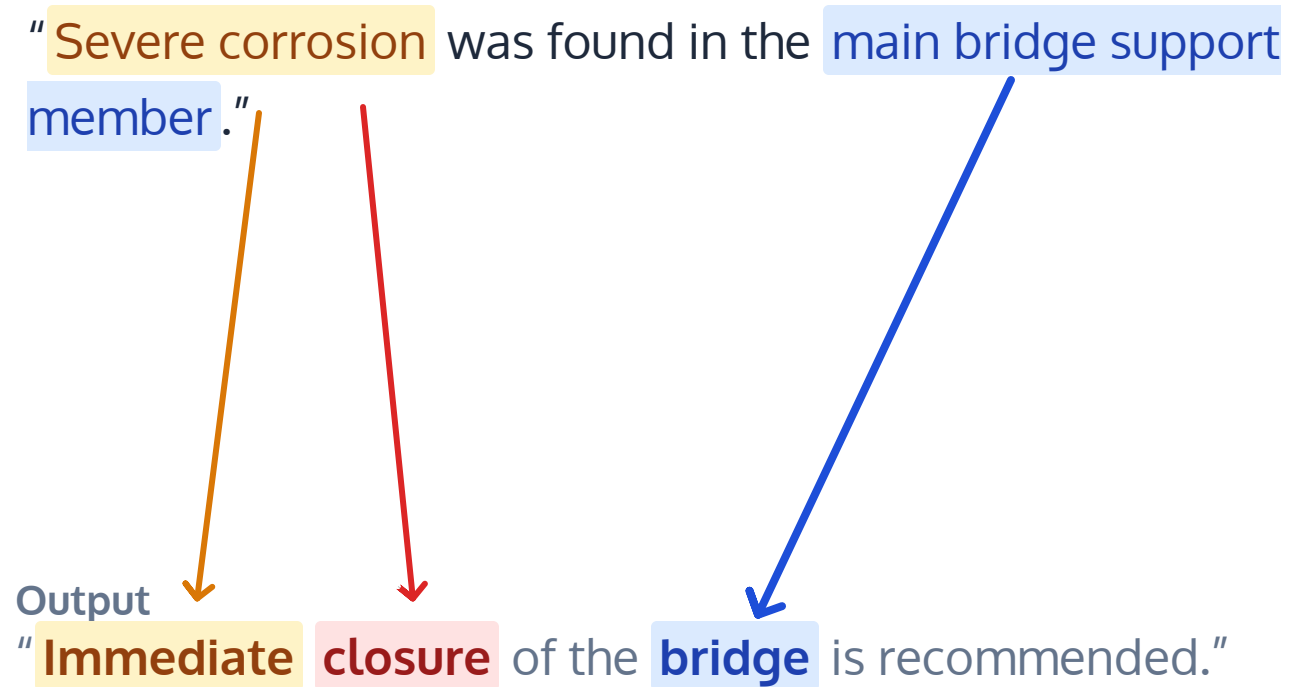
Attention lets the decoder focus on relevant input information



Attention mechanism gives a **larger weight** to more relevant encoder states.

Attention = learned, relevance-weighted retrieval.

Different output words need different parts of the input



When generating ... → the decoder needs ...

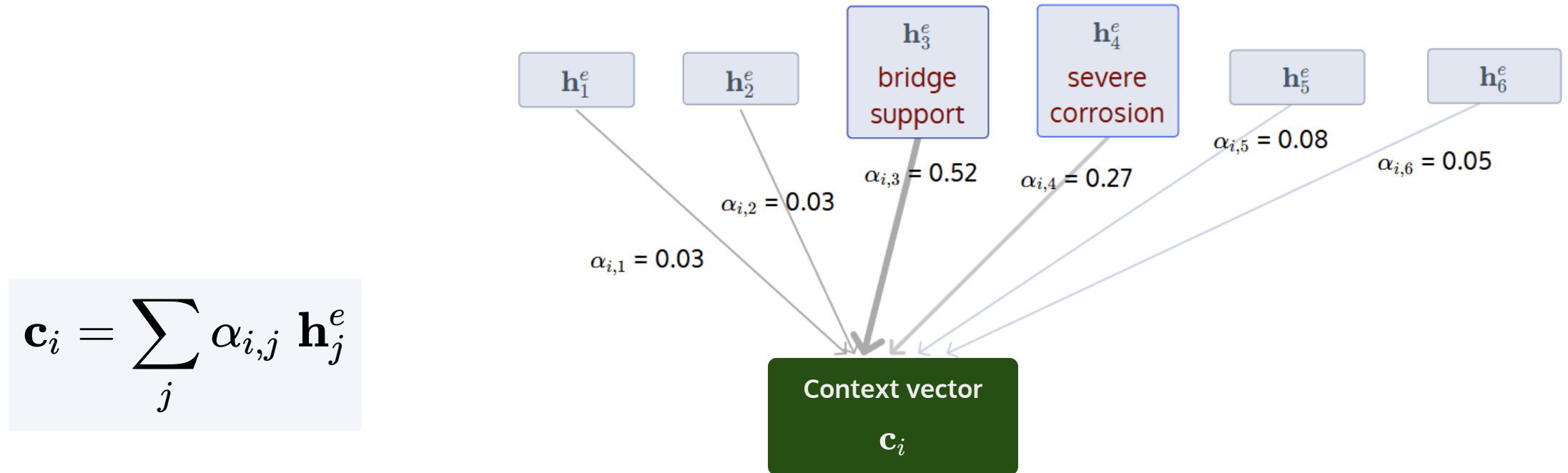
"Immediate" → severe corrosion

"closure" → severe corrosion /
support member

"bridge" → main bridge support
member

The context used by the decoder is not fixed. It changes at every output step.

Attention creates a **context vector** for the current output step



- $\mathbf{h}_j^e$ — encoder representation of input token j
- $\alpha_{i,j}$ — relevance of token j for output step i
- $\mathbf{c}_i$ — context used to predict the next output token

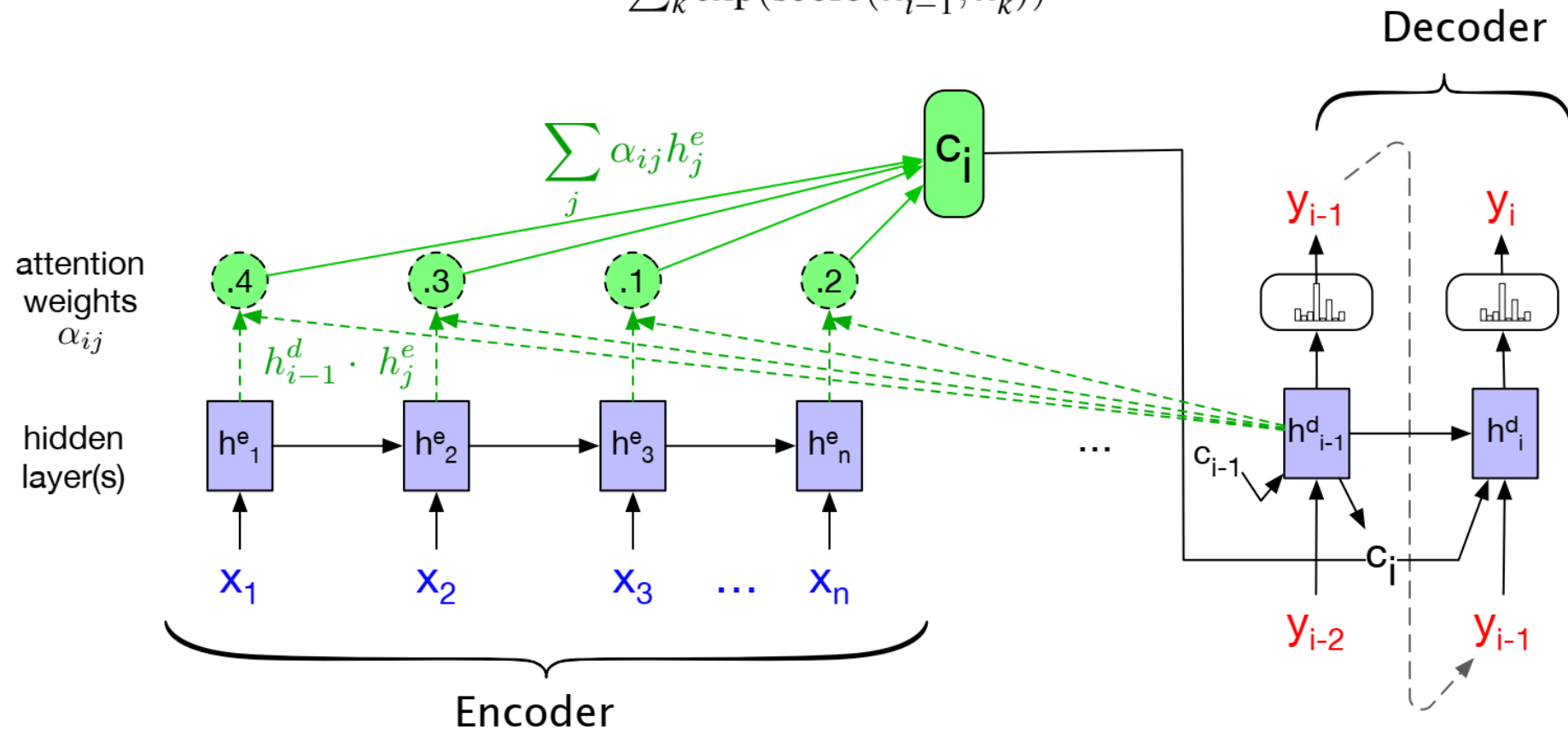
$$\sum_j \alpha_{i,j} = 1 \quad (\text{attention weights sum to one})$$

Computation of a **context vector** for the current output step

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))$$

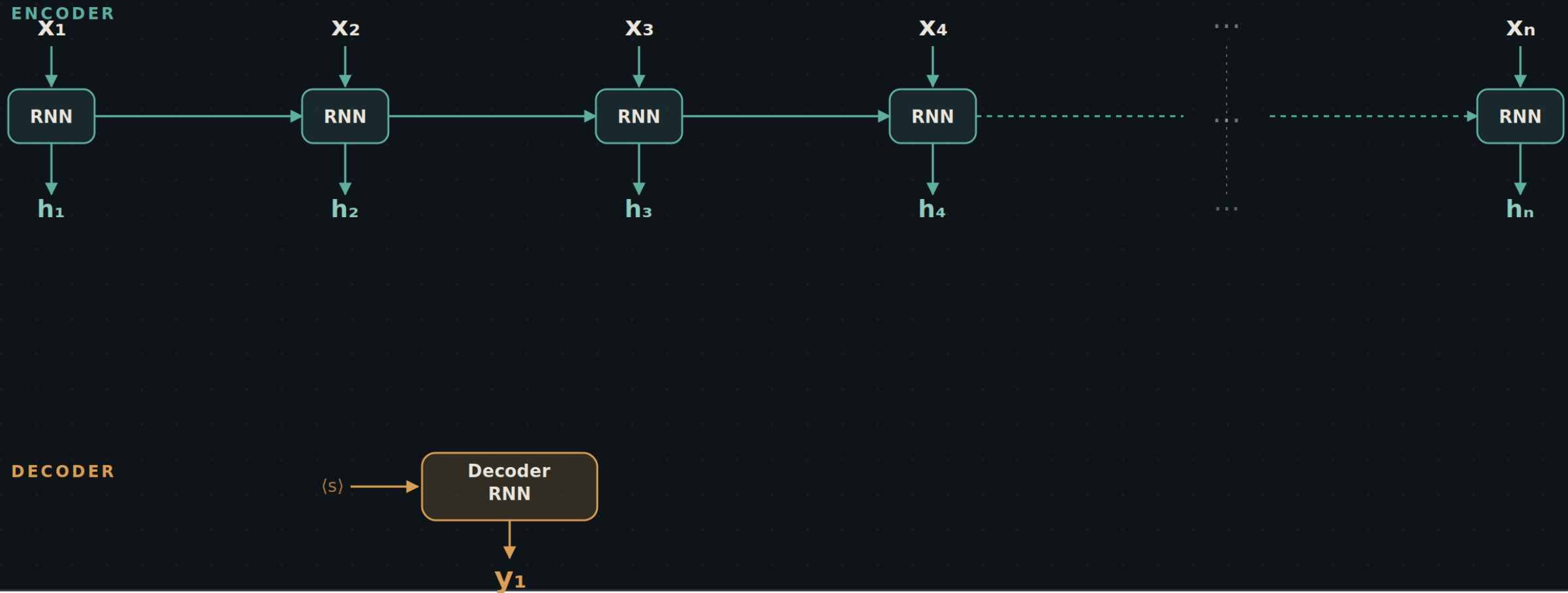
$$= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}$$



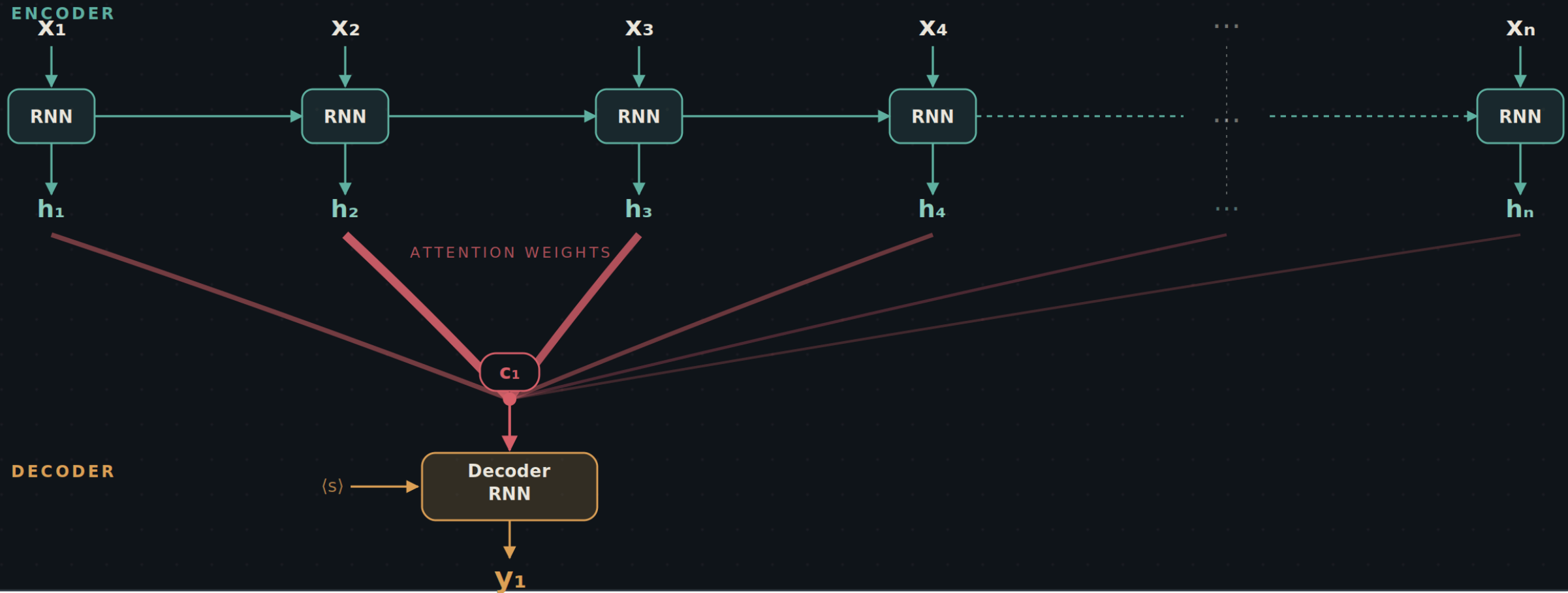
Encoder-Decoder RNN with Attention



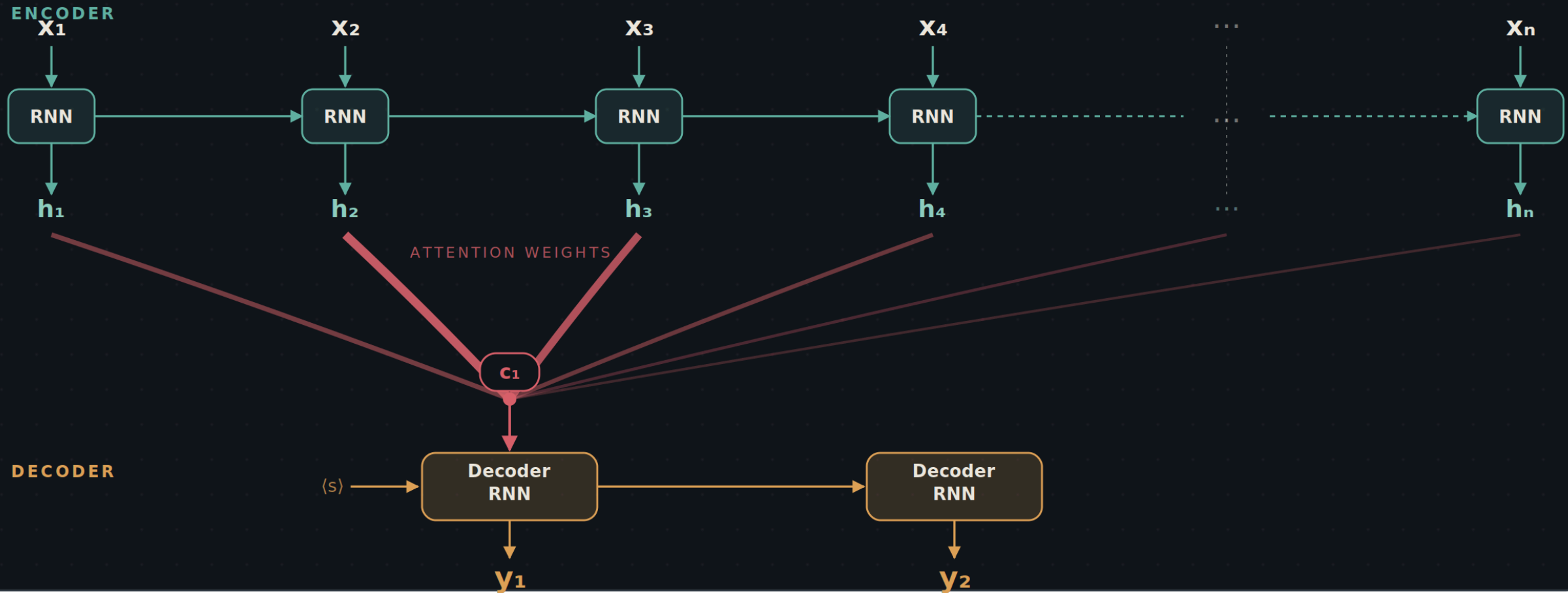
Encoder-Decoder RNN with Attention



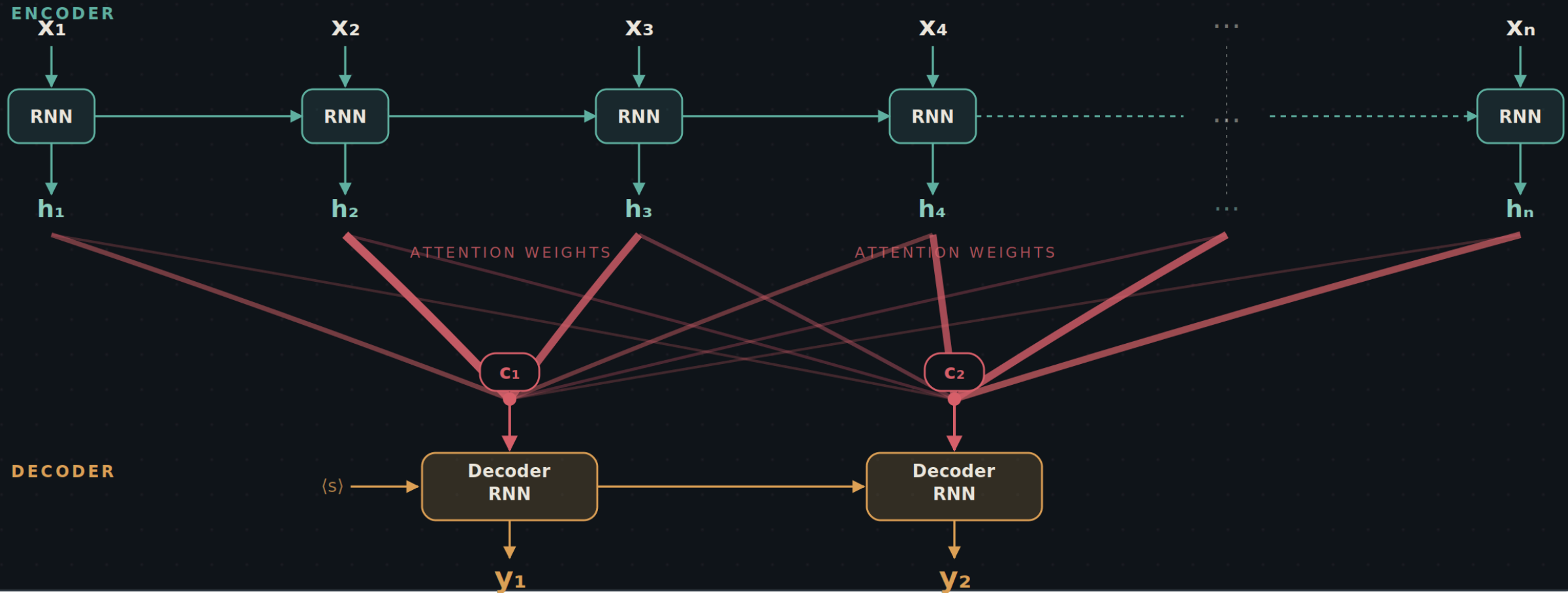
Encoder-Decoder RNN with Attention



Encoder-Decoder RNN with Attention



Encoder-Decoder RNN with Attention



What attention fixes?

Problem in basic encoder–decoder

Entire input compressed into one final vector

Early details may be poorly represented

One fixed summary for every output word

Long input-to-output path

What attention changes

Decoder can access all encoder states

Decoder can revisit early information directly

Each output word gets its own context vector

Direct decoder-to-encoder connections shorten the path

What attention does *not* fix?

Sequential processing is slow

$x_1 \rightarrow [\text{RNN}] \rightarrow h_1 \rightarrow [\text{RNN}] \rightarrow h_2 \rightarrow [\text{RNN}] \rightarrow h_3 \rightarrow$
...
 h_2 needs h_1 · h_3 needs h_2 · h_4 needs h_3

More sequence length = more sequential computation

Distant words have long paths

bridge $\rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_t \rightarrow \text{predict}$

"bridge"

information must travel through every intermediate state

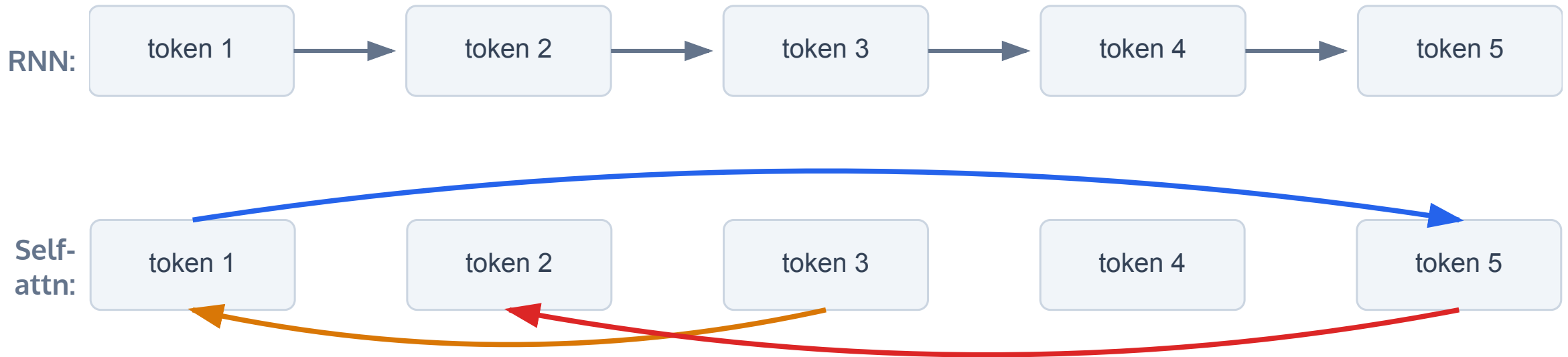
A token 100 words away needs ~ 100 recurrent steps to influence the current state

RNN encoder–decoder:

- limited parallelism during training
- long path between distant pieces of information

What if every token could directly access the earlier tokens that matter?

What if every token could directly interact with other tokens?



Instead of carrying information step by step,
each token can directly retrieve relevant information
from other tokens in the sequence.

This is self-attention.